

DSAA3073: Theories in Data Science

Week 10: Deep Learning I: Approximation Theory & Optimization: Foundations

Concise Learning Sheet

Wei Wang · DSA, HKUST(GZ) · 2026-06-27

Explorative projection

Introduction: The Deep Learning Paradox

Universal approximation (1989) predates deep learning's dominance (2012+). Theory is catching up to practice.

Opening Challenge

Deep learning achieves superhuman performance yet seems to violate classical ML theory:

- **Non-convex optimization** (optimization problems with multiple local minima that are not global minima): Why doesn't gradient descent get stuck in local minima?
- **Overparameterization** (having more model parameters than training samples): Why don't networks with millions of parameters overfit?
- **No explicit regularization** (no manually added penalty terms like weight decay in the loss function): Why do networks generalize without weight decay?

This week: understanding **expressivity** (what functions networks can represent) and **optimization** (how we train them via gradient descent). Next week: **generalization** (why they work on unseen data).

✓ Checkpoint

Learning roadmap:

1. **Expressivity:** What functions can networks represent? (Universal approximation says "any continuous function"; depth vs width shows deep networks are exponentially more efficient)
2. **Optimization:** How do we train them? (Loss landscape analysis; gradient descent convergence despite non-convexity)
3. **NTK regime:** When are networks like kernels? (Infinite-width limit connects to kernel methods from Weeks 6-8)
4. **Implicit regularization:** Why do they generalize? (Gradient descent prefers simple solutions; preview of Week 12's deeper dive)
5. **Double descent:** What are modern generalization phenomena? (Test error has two descents, with peak at interpolation threshold)

Neural Network Basics

Before analyzing deep learning theory, we establish notation. A **feedforward network** (also called a multilayer perceptron) is a function that transforms inputs through multiple layers of computations. With L layers, it computes:

$$f(x; \theta) = W^{(L)} \sigma(W^{(L-1)} \sigma(\dots \sigma(W^{(1)} x + b^{(1)}) \dots) + b^{(L-1)}) + b^{(L)} \quad (1)$$

Here, θ denotes all parameters (all weight matrices $W^{(\ell)}$ and bias vectors $b^{(\ell)}$), and $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is the **activation function** (a nonlinear function applied element-wise). The most common choice is **ReLU** (Rectified Linear Unit): $\sigma(z) = \max(0, z)$, which outputs z if $z > 0$ and 0 otherwise.

Definition: Loss Function Notation

For neural networks, we use the following conventions to connect with the course's established notation from Weeks 2-9:

Neural network notation:

- $\mathcal{L}_{S(\theta)}$ = empirical loss on training set S with parameters θ
- $\mathcal{L}_{D(\theta)}$ = population loss under distribution $\mathcal{D}$ with parameters θ
- $\mathcal{L}_S^* = \inf_{\theta} \mathcal{L}_{S(\theta)}$ = optimal empirical loss

Connection to risk notation:

- $\mathcal{L}_{S(\theta)} = \hat{R}_{S(f_{\theta})}$ where f_{θ} is the network with parameters θ
- $\mathcal{L}_{D(\theta)} = R_{\mathcal{D}}(f_{\theta})$ = expected risk under distribution $\mathcal{D}$

For conciseness, when context is clear (e.g., within a training analysis), we may write $\mathcal{L}$ instead of $\mathcal{L}_S$, but the dataset dependence is always implicit.

Connection

Course connections: We apply tools from Weeks 1-10: PAC learning (2-3), bias-variance (4-5), kernels (6-8), optimization (9-10).

Mental Model: The Three Puzzles of Deep Learning

Modern deep learning violates classical intuitions on three fronts:

Puzzle 1: Non-convex optimization

- Classical intuition: Non-convex problems (problems where the loss function has multiple local minima) are NP-hard, full of bad local minima (points where gradient is zero but loss is far from optimal)
- Reality: Gradient descent reliably finds good solutions
- Resolution (explained this week): Overparameterization creates benign loss landscapes (smooth surfaces with few bad local minima)

Puzzle 2: Overparameterization

- Classical intuition: More parameters than samples ($p > n$) causes overfitting (memorizing training data without learning generalizable patterns)
- Reality: Networks with $p \gg n$ (far more parameters than samples) generalize well
- Resolution (explained this week): Implicit regularization from optimization dynamics (gradient descent naturally prefers simple solutions even without explicit penalties)

Puzzle 3: Interpolation

- Classical intuition: Fitting training data perfectly (zero training error, also called **interpolation**) means overfitting
- Reality: Networks interpolate training data yet generalize to test data
- Resolution (explained this week): Implicit bias toward “simple” interpolating solutions (among many perfect-fit solutions, gradient descent finds smooth ones)

These puzzles connect: overparameterization helps optimization AND generalization.

Universal Approximation Theory

Why study expressivity first? Before we can understand why neural networks generalize or how to train them efficiently, we must answer a more basic question: **what functions can they even represent?** If networks couldn't represent the functions we care about, optimization and generalization would be moot. Universal approximation provides the foundation—networks are flexible enough—but also reveals a puzzle: if shallow networks can approximate any function, why do we need depth?

Theorem: Universal Approximation (Cybenko 1989)

Let σ be a continuous **sigmoidal function** (a smooth S-shaped function like $\sigma(z) = \frac{1}{1+e^{-z}}$ that goes from 0 to 1). Then shallow networks (networks with one hidden layer)

$$f(x) = \sum_{i=1}^n a_i \sigma(w_i^T x + b_i) \quad (2)$$

are **dense** in $C(K)$ (the space of continuous functions on compact set K) for any compact $K \subset \mathbb{R}^d$.

In plain terms: for any continuous function $g : K \rightarrow \mathbb{R}$ and any error tolerance $\varepsilon > 0$, there exists a shallow network f with n neurons such that $\sup_{x \in K} |f(x) - g(x)| < \varepsilon$ (the network approximates g to within ε everywhere on K).

Proof. Proof sketch at the level of ideas:

1. Let S be the **closure** (the set plus all its limit points) of all shallow-network functions in $C(K)$.
2. If $S \neq C(K)$, then by the **Hahn-Banach theorem** (a fundamental result in functional analysis that separates convex sets), there exists a nonzero **bounded linear functional** L (a continuous linear map from functions to real numbers) that vanishes on S (meaning $L(f) = 0$ for all $f \in S$).
3. By the **Riesz representation theorem** (which characterizes linear functionals on continuous functions), we can write $L(f) = \int_K f(x) d\mu(x)$ for some nonzero **signed measure** μ (a generalization of probability measures that can have negative mass).
4. Since all **ridge functions** (functions of the form $\sigma(w^T x + b)$ that depend on x only through a linear projection) lie in S , we would have

$$\int_K \sigma(w^T x + b) d\mu(x) = 0 \quad (3)$$

for every w, b .

5. Cybenko's **discriminatory-function argument** (a technical result showing certain activation functions can distinguish all continuous functions) shows this is impossible for a continuous sigmoidal σ unless $\mu = 0$ (contradicting our assumption that μ is nonzero).

Hence no such separating functional exists, and the network class is dense in $C(K)$. ■

Key Takeaway

Shallow networks are **universal approximators** (they can approximate any continuous function arbitrarily well) but the result is **existential** (it only proves that approximating networks exist, without telling us how to find weights, how many neurons are needed, or how many training samples are required). Analogous universal approximation results hold for ReLU and other non-polynomial activations, under their own technical assumptions.

Wrong Path: Universal Approximation Solves Deep Learning

Why this is incomplete:

1. **Existential, not constructive:** Doesn't tell us how to find the approximating network
2. **No sample complexity:** Doesn't bound generalization error
3. **No computational efficiency:** Doesn't tell us how to train the network
4. **Doesn't explain depth:** Why are deep networks better than shallow ones?

Universal approximation is a starting point, not the full story.

Connection

Connection to Week 2-3 (PAC Learning): Universal approximation addresses **expressivity** (what functions can be represented), but PAC learning addresses **learnability** (what can be learned from finite samples). A hypothesis class can be expressive but not learnable if its sample complexity is too high!

Depth vs. Width: The Power of Composition

But this raises the question: If shallow networks are universal approximators, why does modern deep learning use networks with 50, 100, or even 1000 layers? The answer lies in **efficiency**—while shallow networks can in principle represent any function, they may require exponentially many neurons. Deep networks achieve the same representational power with exponentially fewer parameters by exploiting compositional structure. This efficiency isn't just mathematical elegance—it's the difference between tractable and intractable learning.

Theorem: Depth Separation (Telgarsky 2016; informal)

For ReLU networks, Telgarsky constructed functions representable by deep networks of modest size whose approximation by depth-2 networks (networks with one hidden layer) requires exponentially many units under suitable domains and error metrics. This is an **exponential separation** between depth and width: deep networks with d layers and $O(d)$ neurons can represent functions that require $\Omega(2^d)$ neurons in a shallow network.

Example: Sawtooth Functions

Consider a function with 2^k oscillations (a function that switches between high and low values 2^k times):

- Deep network (depth k): $O(k)$ neurons via composition (building complex patterns by stacking simple transformations)
- Shallow network: $\Omega(2^k)$ neurons (each ReLU creates one “bend” in the function, so we need one neuron per oscillation)

Key insight: Composition (stacking layers, i.e., depth) enables exponential growth in expressiveness; superposition (adding more neurons in one layer, i.e., width) only provides linear growth.

Thought Experiment

Face recognition:

- Shallow: enumerate all pixel combinations (exponentially many)
- Deep: Layer 1 detects edges → Layer 2 combines into parts → Layer 3 assembles faces

Hierarchical composition exponentially reduces parameters needed.

🧠 Mental Model: Depth as Compositional Hierarchy

Think of deep networks as **circuits** rather than lookup tables:

Shallow networks (width): Parallel computation

- Each neuron computes one feature independently
- n neurons $\rightarrow O(n)$ features
- Like OR gates: can represent n things

Deep networks (depth): Sequential composition

- Each layer builds on previous layer's features
- k layers $\rightarrow O(2^k)$ combinations
- Like circuits: exponential combinations from polynomial components

Why this matters:

- Natural functions (images, language) have hierarchical structure
- Deep networks match this structure efficiently
- Shallow networks must “flatten” the hierarchy (exponential blowup)

This is why modern networks are deep (ResNet-50, BERT-24 layers) not wide!

General functions suffer **curse of dimensionality** (exponential scaling with dimension): approximation error scales as $O(n^{-\frac{2}{d}})$ where d is input dimension. Barron class avoids this via Fourier structure (functions whose frequency spectrum decays fast enough).

Approximation Rates

Theorem: Barron's Theorem (1993; simplified)

For functions on a bounded domain with finite **Barron norm** (a measure of function smoothness in frequency space), often written in Fourier form as $C_f = \int |\omega| |\hat{f}(\omega)| d\omega < \infty$ where $\hat{f}(\omega)$ is the Fourier transform of f , suitable shallow networks achieve integrated approximation error $O\left(\frac{C_f}{\sqrt{n}}\right)$ with n neurons. The dimension dependence is hidden in the Barron norm rather than appearing as a generic grid-based curse of dimensionality (the exponential scaling $O(n^{-\frac{2}{d}})$ that typically requires exponentially many parameters in dimension d).

Proof. Intuition for why a Barron-type condition helps:

1. **Fourier representation:** Any sufficiently regular function can be written as $f(x) = \int_{\mathbb{R}^d} \hat{f}(\omega) e^{i\omega^T x} d\omega$ where $\hat{f}(\omega)$ is the Fourier transform (representing the function in frequency space).
2. **Random-feature approximation:** Instead of gridding $\mathbb{R}^d$ (which requires exponentially many points in high dimensions), sample projection directions ω according to a distribution related to $|\omega| |\hat{f}(\omega)|$ (the frequency spectrum weighted by magnitude).
3. **Neural network representation:** Terms depending on one projection $\omega^T x$ (called **ridge functions** because they are constant along ridges perpendicular to ω) can be approximated by functions such as $\sigma(w^T x + b)$ for suitable activations.
4. **Monte Carlo error:** Averaging n sampled ridge functions gives a typical $\frac{1}{\sqrt{n}}$ approximation rate (by the law of large numbers applied to the Monte Carlo approximation of the Fourier integral).

The rigorous theorem requires precise choices of domain, norm, activation, and Barron norm. The key message is that the complexity is controlled by spectral structure (how fast the Fourier transform decays), not by a uniform grid over all d input dimensions. ■

Pause and Think

Question: Why does Barron's theorem avoid the curse of dimensionality?

Hint: Consider how the approximation uses random projections $\omega^T x$ rather than gridding the input space. What does this mean for computational cost?

Answer:

The Fourier spectrum condition (bounded $\int |\omega| |f(\omega)| d\omega$) ensures most function "mass" (the important frequency components) is captured by finitely many projections, regardless of input dimension d . Instead of needing exponentially many grid points (one for each cell in a d -dimensional grid), we need only $O(\frac{d^2}{\epsilon^2})$ random projections.

Key Takeaway

Deep networks achieve better approximation rates for **compositional functions** (functions that can be naturally decomposed into hierarchies of simpler functions, like images composed of edges→parts→objects), translating to exponentially better sample complexity via the **approximation-estimation trade-off** (introduced in Week 4-5: total error = approximation error from model limitations + estimation error from finite samples).

Connection

Preview of Week 12: Approximation theory tells us networks **can** represent functions efficiently, but doesn't explain **generalization**. Week 12 will show how implicit regularization and compression bounds explain why trained networks generalize.

✓ Checkpoint

Expressivity summary:

- Universal approximation: shallow networks can approximate any continuous function (existential)
- Depth separation: deep networks exponentially more efficient for compositional functions
- Approximation rates: Barron's theorem shows dimension-independent rates for suitable function classes

Key insight: Expressivity is necessary but not sufficient—we also need efficient optimization and generalization!

Progress on backbone: We've established that deep networks can represent the functions we need, with depth providing exponential efficiency gains. But representation is only part of the

story—**we still need to find good networks via optimization.** This brings us to the first major puzzle: neural network training is a non-convex optimization problem with millions of parameters and (in theory) exponentially many bad local minima. Yet gradient descent reliably finds excellent solutions. Why?

Optimization Landscape

In a crude random-Hessian heuristic, a critical point in dimension p is unlikely to have all Hessian eigenvalues positive. This intuition suggests why saddles are common, but it is not a theorem about all neural-network losses.

Opening Challenge

Training neural networks requires minimizing the **loss function** $\mathcal{L}_{S(\theta)} = \left(\frac{1}{n}\right) \sum_{i=1}^n \ell(f(x_i; \theta), y_i)$ where ℓ measures prediction error (e.g., squared loss for regression, cross-entropy for classification). This is a non-convex problem (the loss function has multiple local minima) with millions of parameters. Classical theory says finding the global minimum is NP-hard, yet gradient descent works in practice!

Connection

Connection to Week 9-10 (Convex Optimization): For convex functions, every local minimum is global. For non-convex neural networks, this guarantee fails—yet optimization still succeeds. Why?

Critical Points and Loss Geometry

Definition: Critical Points

A point θ^* is **critical** if $\nabla \mathcal{L}_{S(\theta^*)} = 0$ (the gradient vanishes, meaning we cannot immediately improve by moving in any direction). Critical points are classified by the **Hessian** (the matrix of second derivatives) $H = \nabla^2 \mathcal{L}_{S(\theta^*)}$:

- **Local minimum:** $H \succeq 0$ (positive semidefinite: all eigenvalues ≥ 0 , meaning loss increases in all directions)
- **Saddle point:** H has both positive and negative eigenvalues (loss increases in some directions, decreases in others—like a mountain pass)
- **Global minimum:** $\mathcal{L}_{S(\theta^*)} \leq \mathcal{L}_{S(\theta)}$ for all θ (the best possible solution)

Example: Two-Layer Linear Network

Consider a simple case: $f(x) = W_2 W_1 x$ where both layers are linear (no activation function). For squared loss, under standard dimension/rank assumptions, all critical points with $W_2 W_1 = W^*$ (where W^* is an optimal linear predictor) are global minima. These solutions form a **manifold** (a continuous set of equivalent solutions): $(W_1, W_2) = (W^* U^{-1}, U)$ for any compatible invertible matrix U . This shows that even though the parameterization is non-convex (the loss is not convex in W_1, W_2 separately), all critical points that achieve the optimal product are equally good.

Pause and Think

Question: In the linear network example, are there any bad local minima?

Hint: Consider what happens if $W_2 W_1 \neq W^*$. Can this be a local minimum? Think about whether gradient descent can get stuck at such a point.

Answer:

For deep linear networks with squared loss, under standard full-rank/dimension assumptions, every local minimum is a global minimum. The loss is convex in the product $W = W_2 W_1$ (meaning if we fix the product, changing the factorization doesn't change the loss), although proving the factorized non-convex parameterization has no bad local minima requires additional rank assumptions. Bad critical points exist, but they are saddle points (with negative Hessian eigenvalues), not local minima.

Proof Template: Analyzing Loss Landscape

To show a neural network has benign loss landscape:

Step 1: Identify critical points

- Solve $\nabla \mathcal{L}_{S(\theta)} = 0$
- Characterize the solution set

Step 2: Compute Hessian

- Calculate $H = \nabla^2 \mathcal{L}_{S(\theta)}$
- Analyze eigenvalues

Step 3: Classify critical points

- Local minima: all eigenvalues ≥ 0
- Saddles: mixed eigenvalues
- Show no bad local minima (all local minima have loss close to global minimum)

Step 4: Show gradient descent escapes saddles

- Negative eigenvalue $\rightarrow$ exponential divergence in that direction
- Noise (from SGD) ensures escape with high probability

Example: Linear networks

- Critical points: $W_2 W_1 = W^*$ (global minima) or rank-deficient (saddles)
- No bad local minima!

Overparameterization Helps Optimization

Definition: Overparameterization Regimes

Neural networks can be categorized by the ratio of parameters p to training samples n :

- **Underparameterized:** $p < n$ (classical regime: fewer parameters than samples, cannot perfectly fit training data)
- **Critically parameterized:** $p \approx n$ (interpolation threshold: just enough parameters to fit training data, typically one or very few solutions)
- **Overparameterized:** $p \gg n$ (modern deep learning: far more parameters than samples, many solutions that fit training data perfectly)

Author's Take

Empirical observations for wide networks:

1. Few bad local minima (loss near global minimum)
2. Many saddle points, but gradient descent escapes them
3. Mode connectivity: different minima connected by low-loss paths
4. Easier optimization as width increases

Theorem: Loss Landscape Structure (Choromanska et al. 2015, informal)

For random neural networks, under certain statistical-physics-inspired assumptions, local minima (critical points where all Hessian eigenvalues are positive) tend to have loss close to the global minimum $\mathcal{L}_S^*$. This means “bad” local minima (with loss much higher than optimal) are rare. The analysis is inspired by **spin glass theory** from statistical physics (the study of disordered magnetic systems with many metastable states).

 Mental Model: High-Dimensional Optimization Intuition

Why is high-dimensional optimization easier than low-dimensional?

Low dimension (e.g., $d = 2$):

- Critical points can be surrounded by barriers
- Local minima can trap optimization
- Hard to escape

High dimension (e.g., $d = 10^6$):

- To be a local minimum, all d Hessian eigenvalues must be positive
- Probability decreases exponentially with d (for random-like Hessians)
- Most critical points are saddles
- Saddles have escape directions (negative eigenvalues)

Key insight: In high dimensions, it's hard to be trapped—there's almost always a way out!

Caveat: This is intuition, not theorem. Real neural networks have structure beyond random matrices.

Gradient Descent Convergence

This theorem only guarantees finding a **stationary point** ($\nabla \mathcal{L}_S = 0$, where the gradient vanishes), which could be a saddle point rather than a local minimum. We need additional analysis (like the next theorem) to show that gradient descent escapes saddles.

Theorem: Gradient Descent for Smooth Non-Convex Functions

Assume:

- **Loss function:** $\mathcal{L}_S : \mathbb{R}^p \rightarrow \mathbb{R}$ is the empirical loss on training data.
- **Smoothness:** $\mathcal{L}_S$ is L -smooth: $\|\nabla \mathcal{L}_{S(\theta)} - \nabla \mathcal{L}_{S(\theta')}\|_2 \leq L\|\theta - \theta'\|_2$ for all θ, θ'
- **Algorithm:** Gradient descent with $\theta_{t+1} = \theta_t - \eta \nabla \mathcal{L}_{S(\theta_t)}$
- **Step size:** $\eta \leq \frac{1}{L}$
- **Lower bound:** $\mathcal{L}_{S(\theta)} \geq \mathcal{L}_S^*$ for all θ (where $\mathcal{L}_S^* = \inf_{\theta} \mathcal{L}_{S(\theta)}$)

Then: After T iterations, the minimum gradient norm satisfies:

$$\min_{t=0, \dots, T-1} \|\nabla \mathcal{L}_{S(\theta_t)}\|_2^2 \leq \frac{2(\mathcal{L}_{S(\theta_0)} - \mathcal{L}_S^*)}{\eta T} \quad (4)$$

Interpretation: In $O(\frac{1}{\varepsilon^2})$ iterations, GD finds a point with gradient norm $\leq \varepsilon$, guaranteeing convergence to a first-order stationary point (where $\nabla \mathcal{L}_S = 0$).

← From Week 9: Online Convex Optimization

Week 9 introduced smoothness and the descent lemma for convex functions. The same analysis applies to non-convex neural network losses, guaranteeing convergence to stationary points (though not necessarily global minima).

Proof. By L -smoothness (recall the descent lemma from Week 9), substituting $\theta_{t+1} = \theta_t - \eta \nabla \mathcal{L}_{S(\theta_t)}$:

$$\begin{aligned} \mathcal{L}_{S(\theta_{t+1})} &\leq \mathcal{L}_{S(\theta_t)} - \eta \|\nabla \mathcal{L}_{S(\theta_t)}\|_2^2 + \frac{\eta^2 L}{2} \|\nabla \mathcal{L}_{S(\theta_t)}\|_2^2 \\ &= \mathcal{L}_{S(\theta_t)} - \eta \left(1 - \frac{\eta L}{2}\right) \|\nabla \mathcal{L}_{S(\theta_t)}\|_2^2 \end{aligned} \quad (5)$$

For $\eta \leq \frac{1}{L}$, we have $1 - \frac{\eta L}{2} \geq \frac{1}{2}$, so:

$$\mathcal{L}_{S(\theta_{t+1})} \leq \mathcal{L}_{S(\theta_t)} - \frac{\eta}{2} \|\nabla \mathcal{L}_{S(\theta_t)}\|_2^2 \quad (6)$$

Summing from $t = 0$ to $T - 1$:

$$\sum_{t=0}^{T-1} \|\nabla \mathcal{L}_{S(\theta_t)}\|_2^2 \leq \frac{2}{\eta} (\mathcal{L}_{S(\theta_0)} - \mathcal{L}_{S(\theta_T)}) \leq \frac{2}{\eta} (\mathcal{L}_{S(\theta_0)} - \mathcal{L}_S^*) \quad (7)$$

Therefore:

$$\min_{t=0, \dots, T-1} \|\nabla \mathcal{L}_{S(\theta_t)}\|_2^2 \leq \frac{2(\mathcal{L}_{S(\theta_0)} - \mathcal{L}_S^*)}{\eta T} \quad (8)$$

■

Theorem: Escaping Strict Saddle Points (Jin et al. 2017)**Assume:**

- **Smoothness:** $\mathcal{L}_S$ is L -smooth
- **Hessian-Lipschitz:** $\mathcal{L}_S$ is ρ -Hessian-Lipschitz: $\|\nabla^2 \mathcal{L}_{S(\theta)} - \nabla^2 \mathcal{L}_{S(\theta')}\| \leq \rho \|\theta - \theta'\|$ for all θ, θ'
- **Strict-saddle property:** Every critical point θ (where $\nabla \mathcal{L}_{S(\theta)} = 0$) is either:
 - Local minimum: $\lambda_{\min}(\nabla^2 \mathcal{L}_{S(\theta)}) \geq 0$, or
 - Strict saddle: $\lambda_{\min}(\nabla^2 \mathcal{L}_{S(\theta)}) \leq -\gamma$ for some $\gamma > 0$
- **Bounded domain:** $\|\theta\|_2 \leq D$ for all iterates
- **Perturbed GD:** Add Gaussian noise $\xi_t \sim \mathcal{N}(0, \sigma^2 I)$ every τ iterations
- **Hyperparameters:** Step size $\eta = O(\frac{1}{L})$, noise level $\sigma = \Omega(\frac{\gamma}{\rho})$

Then: With probability $\geq 1 - \delta$, perturbed gradient descent reaches an ε -second-order stationary point in $\text{poly}(d, \frac{1}{\varepsilon}, \log(\frac{1}{\delta}))$ iterations.

An ε -second-order stationary point satisfies:

- $\|\nabla \mathcal{L}_{S(\theta)}\| \leq \varepsilon$ (nearly zero gradient), and
- $\lambda_{\min}(\nabla^2 \mathcal{L}_{S(\theta)}) \geq -\sqrt{\varepsilon}$ (no large negative eigenvalues)

Noise is crucial! Mini-batch sampling in SGD (stochastic gradient descent, which computes gradients on random subsets of data) provides implicit noise that helps escape saddles. Pure gradient descent (full-batch) without noise can get stuck at saddle points.

Example: Escaping a Saddle Point

Consider $\mathcal{L}_{S(\theta_1, \theta_2)} = \theta_1^2 - \theta_2^2$ (saddle at origin).

Gradient flow from $(\varepsilon, 0)$:

$$\begin{aligned}\theta_1(t) &= \varepsilon e^{-2t} \rightarrow 0 \\ \theta_2(t) &= 0\end{aligned}\tag{9}$$

Converges to the saddle point!

Stochastic GD with noise:

$$\begin{aligned}\theta_1(t+1) &= \theta_1(t) - \eta \cdot 2\theta_1(t) + \xi_1(t) \\ \theta_2(t+1) &= \theta_2(t) + \eta \cdot 2\theta_2(t) + \xi_2(t)\end{aligned}\tag{10}$$

where ξ_1, ξ_2 are noise terms. If $\xi_2(0) \neq 0$, then θ_2 grows exponentially and escapes!

✓ Checkpoint**Optimization landscape insights:**

1. Non-convex optimization is hard in general, but neural networks have special structure
2. Overparameterization creates benign landscapes (few bad local minima)
3. High dimension helps: most critical points are saddles, not local minima
4. Gradient descent finds stationary points; SGD noise helps escape saddles

Key question: Can we get stronger guarantees than just “stationary point”?

Polyak-Łojasiewicz Condition

The **Polyak-Łojasiewicz (PL) condition** bridges the gap between optimization. It is a property that, when satisfied, guarantees fast convex functions.

PL is weaker than **strong convexity** (introduced in Week 9: $\mathcal{L}(y) \geq \mathcal{L}(x) + \nabla \mathcal{L}(x)^T(y - x) + (\frac{\mu}{2})\|y - x\|_2^2$) but stronger than mere smoothness. It's a "Goldilocks" condition for non-convex functions: just enough structure to guarantee fast convergence without requiring convexity.

Definition: Polyak-Łojasiewicz (PL) Condition

A function $\mathcal{L} : \mathbb{R}^p \rightarrow \mathbb{R}$ satisfies the **Polyak-Łojasiewicz (PL) condition** with parameter $\mu > 0$ if, for every point $\theta \in \mathbb{R}^p$,

$$\|\nabla \mathcal{L}(\theta)\|_2^2 \geq 2\mu(\mathcal{L}(\theta) - \mathcal{L}^*) \quad (11)$$

where $\mathcal{L}^* = \inf_{\theta \in \mathbb{R}^p} \mathcal{L}(\theta)$ is the global minimum value.

Interpretation: The squared gradient norm is always proportional to the suboptimality gap $\mathcal{L}(\theta) - \mathcal{L}^*$. When $\mathcal{L}(\theta) > \mathcal{L}^*$, the gradient must be bounded away from zero, ensuring a strong descent direction.

Scope and applicability:

- For neural networks, the PL condition may hold for the **empirical loss** $\mathcal{L}_{S(\theta)}$ in the **overparameterized regime** (when number of parameters $p \gg$ number of samples n)
- The PL condition typically does **NOT** hold for the **population loss** $\mathcal{L}_{D(\theta)}$, which may have multiple local minima
- When PL holds for $\mathcal{L}_S$, gradient descent on training loss converges exponentially fast to zero training error, even though the loss landscape may be non-convex

Key property: PL is weaker than strong convexity but stronger than general non-convexity. It allows non-convex loss surfaces while still guaranteeing global convergence.

Theorem: Exponential Convergence under PL

If $\mathcal{L}_S$ is L -smooth, satisfies PL with parameter μ , and gradient descent uses $\eta \leq \frac{1}{L}$, then:

$$\mathcal{L}_{S(\theta_t)} - \mathcal{L}_S^* \leq (1 - \eta\mu)^t (\mathcal{L}_{S(\theta_0)} - \mathcal{L}_S^*) \quad (12)$$

Proof. By smoothness and the gradient descent update:

$$\begin{aligned} \mathcal{L}_{S(\theta_{t+1})} &\leq \mathcal{L}_{S(\theta_t)} - \eta \|\nabla \mathcal{L}_{S(\theta_t)}\|_2^2 + \frac{\eta^2 L}{2} \|\nabla \mathcal{L}_{S(\theta_t)}\|_2^2 \\ &= \mathcal{L}_{S(\theta_t)} - \eta \left(1 - \frac{\eta L}{2}\right) \|\nabla \mathcal{L}_{S(\theta_t)}\|_2^2 \end{aligned} \quad (13)$$

For $\eta \leq \frac{1}{L}$, we have $1 - \frac{\eta L}{2} \geq \frac{1}{2}$, so:

$$\mathcal{L}_{S(\theta_{t+1})} \leq \mathcal{L}_{S(\theta_t)} - \frac{\eta}{2} \|\nabla \mathcal{L}_{S(\theta_t)}\|_2^2 \quad (14)$$

By the PL condition:

$$\begin{aligned}\mathcal{L}_{S(\theta_{t+1})} &\leq \mathcal{L}_{S(\theta_t)} - \eta\mu(\mathcal{L}_{S(\theta_t)} - \mathcal{L}_S^*) \\ &= (1 - \eta\mu)\mathcal{L}_{S(\theta_t)} + \eta\mu\mathcal{L}_S^*\end{aligned}\tag{15}$$

Subtracting $\mathcal{L}_S^*$ from both sides:

$$\mathcal{L}_{S(\theta_{t+1})} - \mathcal{L}_S^* \leq (1 - \eta\mu)(\mathcal{L}_{S(\theta_t)} - \mathcal{L}_S^*)\tag{16}$$

Iterating gives the result. ■

Key Takeaway

The PL condition is weaker than convexity (doesn't require the function to be bowl-shaped) but still gives exponential convergence (loss decays exponentially fast with iterations). Some overparameterized networks satisfy PL, explaining their convex-like optimization behavior despite being non-convex functions.

Example: Linear Regression Satisfies PL

For linear regression with squared loss:

$$\mathcal{L}_{S(\theta)} = \frac{1}{2n} \sum_{i=1}^n (y_i - \theta^T x_i)^2 = \frac{1}{2n} \|y - X\theta\|_2^2\tag{17}$$

where $X \in \mathbb{R}^{n \times p}$ is the data matrix with rows $x_1^T, \dots, x_n^T$.

Case 1: Full column rank ($p \leq n$ and $\text{rank}(X) = p$)

- $\mathcal{L}_S$ satisfies PL with $\mu = \lambda_{\min} \frac{X^T X}{n} > 0$
- Unique minimizer: $\theta^* = (X^T X)^{-1} X^T y$
- Gradient descent converges exponentially: $\mathcal{L}_{S(\theta_t)} - \mathcal{L}_S^* \leq (1 - \eta\mu)^t (\mathcal{L}_{S(\theta_0)} - \mathcal{L}_S^*)$

Case 2: Rank deficient ($\text{rank}(X) < p$)

- Multiple minimizers exist (infinitely many solutions to $X\theta = y$)
- PL holds on the identifiable subspace (column space of X^T) with $\mu = \lambda_{\min}^+ \frac{X^T X}{n}$ where $\lambda_{\min}^+$ is the smallest positive eigenvalue
- Gradient descent from $\theta_0 = 0$ converges to the minimum-norm solution: $\theta^* = X^T (X X^T)^{-1} y$

Connection to neural networks: Overparameterized networks behave like rank-deficient linear regression—many parameter configurations achieve zero training loss, and gradient descent finds the minimum-norm solution.

Connection

Connection to Week 9 (Convex Optimization):

- Convex + smooth $\rightarrow O(\frac{1}{T})$ convergence (first-order stationary point)
- Strongly convex + smooth $\rightarrow$ exponential convergence
- Non-convex + PL $\rightarrow$ exponential convergence (mimics strong convexity without convexity!)

PL is the key bridge between convex and neural network optimization.

Proof Template: Proving PL Condition

To show function $\mathcal{L}$ satisfies PL with parameter μ :

Step 1: Lower bound gradient norm

- Express $\|\nabla \mathcal{L}(\theta)\|_2^2$ explicitly
- Find relationship to suboptimality $\mathcal{L}(\theta) - \mathcal{L}^*$

Step 2: Use problem structure

- For least squares: use smallest eigenvalue of Gram matrix
- For overparameterized networks: use Gram matrix of network Jacobian
- Key: relate gradient to distance from optimum

Step 3: Verify the inequality

- Show $\|\nabla \mathcal{L}(\theta)\|_2^2 \geq 2\mu(\mathcal{L}(\theta) - \mathcal{L}^*)$
- Identify μ (usually involves smallest eigenvalue)

Example: Linear regression

- $\nabla \mathcal{L}_{S(\theta)} = -\frac{1}{n} X^T (y - X\theta)$
- $\|\nabla \mathcal{L}_{S(\theta)}\|_2^2 = \frac{1}{n^2} \|X^T (y - X\theta)\|_2^2 \geq \frac{\lambda_{\min}(X^T X)}{n^2} \|y - X\theta\|_2^2$
- $\mathcal{L}_{S(\theta)} - \mathcal{L}_S^* = \frac{1}{2n} \|y - X\theta\|_2^2 - \frac{1}{2n} \|y - X\theta^*\|_2^2$
- Under conditions, get PL with $\mu = \lambda_{\min} \frac{X^T X}{n}$

Now we're ready to answer: How can we rigorously analyze neural network training dynamics? Empirical observations about loss landscapes are valuable, but we seek mathematical guarantees. The Neural Tangent Kernel provides an elegant answer: in the infinite-width limit, neural networks become linear models with a fixed kernel, and we can leverage all the theory from kernel methods (Weeks 6-8) to prove convergence and generalization bounds. This is our first rigorous handle on why gradient descent works.

Neural Tangent Kernel Theory

This section connects neural networks to kernel methods from Week 10. For very wide neural networks, wide neural networks behave almost like kernel methods.

Opening Challenge

For very wide networks trained with small learning rates, training is linear! This surprising connection links neural networks to kernel methods, providing theoretical guarantees for optimization.

From kernel to matrix

The NTK kernel $\Theta(x, x')$ is a function on input pairs. For training on dataset $S = \{x_1, \dots, x_n\}$, we compute the **NTK Gram matrix**:

$$\Theta_S = \begin{pmatrix} \Theta(x_1, x_1) & \dots & \Theta(x_1, x_n) \\ \vdots & \ddots & \vdots \\ \Theta(x_n, x_1) & \dots & \Theta(x_n, x_n) \end{pmatrix} \in \mathbb{R}^{n \times n}$$

This matrix governs the training dynamics: $\dot{\mathbf{f}} = -\Theta_S(\mathbf{f} - \mathbf{y})$ where $\mathbf{f} = (f(x_1; \theta), \dots, f(x_n; \theta))$.

Connection

Connection to Week 11 Background Section (RKHS): Recall that kernels measure similarity $k(x, x')$ between inputs and define a **reproducing kernel Hilbert space (RKHS)** where learning takes place. The Neural Tangent Kernel plays the same role for infinite-width networks, connecting deep learning to kernel theory.

Linearization and the NTK

Definition: Neural Tangent Kernel

The **Neural Tangent Kernel (NTK)** at initialization θ_0 is:

$$\Theta(x, x') = \nabla_{\theta} f(x; \theta_0)^T \nabla_{\theta} f(x'; \theta_0) = \sum_{i=1}^p \frac{\partial f(x; \theta_0)}{\partial \theta_i} \frac{\partial f(x'; \theta_0)}{\partial \theta_i} \quad (18)$$

This measures the alignment between parameter gradients at two inputs x and x' . Intuitively, if $\Theta(x, x')$ is large, changing parameters to fit x will also change the output at x' (the two inputs are similar from the network's perspective).

Example: NTK for a Linear Model

For the ordinary linear model $f_w(x) = w^T x$, the parameter gradient is $\nabla_w f_w(x) = x$. Therefore the NTK is

$$\Theta(x, x') = x^T x', \quad (20)$$

exactly the standard linear kernel. This is why parameter gradients can be viewed as features.

Example: NTK for Two-Layer Network

For a two-layer network $f(x) = \left(\frac{1}{\sqrt{m}}\right) \sum_{i=1}^m a_i \sigma(w_i^T x)$ with $a_i \in \{-1, +1\}$ fixed (only training the input weights w_i):

$$\Theta(x, x') = \frac{1}{m} \sum_{i=1}^m \sigma'(w_i^T x) \sigma'(w_i^T x') x^T x' \quad (21)$$

where σ' is the derivative of the activation function.

As width $m \rightarrow \infty$, by the **law of large numbers** (the average of many independent random variables converges to their expectation):

$$\Theta(x, x') \rightarrow \mathbb{E}_{w \sim \mathcal{N}(0, I)} [\sigma'(w^T x) \sigma'(w^T x')] x^T x' \quad (22)$$

The NTK becomes deterministic! The randomness from initialization averages out.

Note: Because a_i are fixed, only the input-weight gradients contribute. If output weights are also trained, the standard two-layer NTK has an additional covariance term from the a_i gradients.

Pause and Think

Question: Why does the NTK become deterministic as width increases?

Hint: Think about the law of large numbers. What happens to the average of many i.i.d. (independent and identically distributed) random variables?

Answer:

The NTK is an average over m neurons: $\Theta = \left(\frac{1}{m}\right) \sum_{i=1}^m$ contribution from neuron i . Each neuron is initialized independently with random weights. As $m \rightarrow \infty$, this average concentrates around its expectation by the law of large numbers (the sample average converges to the population mean), becoming deterministic. The randomness from initialization averages out.

Example: Computing NTK for ReLU

For ReLU activation $\sigma(z) = \max(0, z)$, the derivative is $\sigma'(z) = \mathbb{1}_{z>0}$ (the indicator function: 1 if $z > 0$, and 0 otherwise) away from $z = 0$. The value at $z = 0$ is irrelevant under continuous Gaussian initialization because it occurs with probability zero.

The NTK becomes:

$$\Theta(x, x') = \mathbb{E}_{w \sim \mathcal{N}(0, I)} [\mathbb{1}_{w^T x > 0} \mathbb{1}_{w^T x' > 0}] x^T x' \quad (23)$$

The expectation is the probability that both $w^T x$ and $w^T x'$ are positive (both projections land in the positive half-space):

$$\mathbb{E}[\mathbb{1}_{w^T x > 0} \mathbb{1}_{w^T x' > 0}] = \mathbb{P}(w^T x > 0 \text{ and } w^T x' > 0) \quad (24)$$

For Gaussian w , this depends on the angle θ between x and x' . Using geometric arguments from probability theory:

$$\mathbb{P}(w^T x > 0 \text{ and } w^T x' > 0) = \frac{\pi - \theta}{2\pi} = \frac{1}{2} - \frac{\theta}{2\pi} \quad (25)$$

where $\cos \theta = \frac{x^T x'}{\|x\|_2 \|x'\|_2}$ is the normalized dot product.

Therefore, the ReLU NTK has a closed form:

$$\Theta(x, x') = \left(\frac{\pi - \theta}{2\pi} \right) x^T x' = \left(\frac{1}{2} - \frac{\arccos\left(\frac{x^T x'}{\|x\|_2 \|x'\|_2}\right)}{2\pi} \right) x^T x' \quad (26)$$

This is a specific kernel determined entirely by the ReLU activation and Gaussian initialization—no learning involved!

🧠 Mental Model: NTK as Feature Learning Frozen at Initialization

Understanding the NTK regime:

Standard neural network training:

- Parameters θ change significantly during training
- Features $\varphi(x) = \nabla_{\theta} f(x; \theta)$ evolve (feature learning)
- Network output $f(x; \theta)$ is highly nonlinear in parameters

NTK regime (infinite width):

- Parameters change infinitesimally (each neuron's contribution vanishes as $\frac{1}{\sqrt{m}}$)
- Features $\varphi(x) \approx \nabla_{\theta} f(x; \theta_0)$ stay nearly constant
- Training becomes **linear** in features frozen at initialization
- Equivalent to kernel regression with kernel $\Theta(x, x') = \varphi(x)^T \varphi(x')$

Key insight: Infinite width “freezes” the features at random initialization. Training only adjusts coefficients, not features. This is why NTK connects to kernel methods (Week 11 Background Section)!

Limitation: Real networks (finite width) learn features. NTK explains optimization but misses representation learning.

Theorem: NTK Training Dynamics (Jacot et al. 2018; simplified)

With squared loss $\mathcal{L}_{S(\theta)} = \frac{1}{2n} \sum_i (f(x_i; \theta) - y_i)^2$, the infinite-width gradient-flow dynamics are:

$$\frac{d\mathbf{f}_t}{dt} = -\frac{1}{n} \Theta(\mathbf{f}_t - \mathbf{y}) \quad (27)$$

where $\mathbf{f}_t = (f(x_1; \theta_t), \dots, f(x_n; \theta_t))^T$ is the vector of network outputs on training data, $\mathbf{y} = (y_1, \dots, y_n)^T$ is the target vector, and $\Theta_{ij} = \Theta(x_i, x_j)$ is the $n \times n$ **Gram matrix** of the NTK evaluated on training inputs. This is a **linear** ordinary differential equation (ODE) under the constant-NTK approximation—unlike the original non-linear neural network dynamics!

Proof. Sketch of the key steps:

1. Gradient descent update:

$$\frac{d\theta_t}{dt} = -\nabla_{\theta} \mathcal{L}_{S(\theta_t)} = \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i; \theta_t)) \nabla_{\theta} f(x_i; \theta_t) \quad (28)$$

2. Network output evolution:

$$\begin{aligned} \frac{df(x; \theta_t)}{dt} &= \nabla_{\theta} f(x; \theta_t)^T \frac{d\theta_t}{dt} \\ &= \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i; \theta_t)) \nabla_{\theta} f(x; \theta_t)^T \nabla_{\theta} f(x_i; \theta_t) \\ &= \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i; \theta_t)) \Theta_t(x, x_i) \end{aligned} \quad (29)$$

3. **Infinite-width limit:** As width $m \rightarrow \infty$, on a fixed finite dataset and over suitable training horizons, the NTK remains approximately constant: $\Theta_t(x, x') \approx \Theta_0(x, x') = \Theta(x, x')$.
4. This gives the linear dynamics: $\frac{df_t}{dt} = -\frac{1}{n}\Theta(f_t - y)$.

■

Theorem: Exponential Convergence in NTK Regime

If Θ has smallest eigenvalue $\lambda_{\min} > 0$:

$$\|f_t - y\|_2 \leq \exp\left(-\frac{\lambda_{\min} t}{n}\right) \|f_0 - y\|_2 \quad (30)$$

Proof. The linear ODE $\frac{df_t}{dt} = -\frac{1}{n}\Theta(f_t - y)$ has solution:

$$f_t - y = \exp\left(-\frac{\Theta t}{n}\right) (f_0 - y) \quad (31)$$

Since Θ is positive definite:

$$\|f_t - y\|_2 \leq \left\| \exp\left(-\frac{\Theta t}{n}\right) \right\|_2 \|f_0 - y\|_2 \leq \exp\left(-\frac{\lambda_{\min} t}{n}\right) \|f_0 - y\|_2 \quad (32)$$

■

Key Takeaway

In the infinite-width limit, neural network training behaves like **kernel gradient flow** or **kernel interpolation** in the NTK RKHS (reproducing kernel Hilbert space associated with kernel Θ); with explicit ridge regularization, the predictor corresponds to kernel ridge regression. Dynamics are linear under the constant-NTK approximation and converge exponentially when the empirical NTK Gram matrix Θ is positive definite (all eigenvalues positive).

Connection

Week 6-8 connection: The NTK plays the role of kernel $k(x, x')$ in kernel gradient flow/interpolation (the dynamics of continuously optimizing in the RKHS); with ridge regularization (adding $\lambda \|f\|_{\mathcal{H}}^2$ penalty), it connects to kernel ridge regression (introduced in Week 7). But unlike hand-designed kernels (polynomial, RBF), the NTK is determined by architecture and initialization—it's not chosen a priori but emerges from the network structure.

✓ **Checkpoint**

Original proof requires $m = \Omega(n^6)$ (impractical). Recent work improves to $m = O(n^2)$ or $m = O(n)$.

NTK theory summary:

1. Infinite-width networks behave like kernel methods with kernel $\Theta(x, x')$ determined by architecture and initialization
2. Training dynamics become **linear** (gradient flow in RKHS, unlike the original non-linear neural network dynamics)
3. Convergence is exponential when the NTK Gram matrix is positive definite (like kernel ridge regression from Week 7)
4. NTK is determined by architecture and initialization, not learned during training (features are frozen at initialization)

Key limitation: NTK assumes features don't change during training (lazy training regime). Real finite-width networks learn features—this is their main advantage over kernel methods!

Lazy Training vs. Feature Learning

Definition: Training Regimes

Neural networks exhibit different behaviors depending on width:

- **Lazy training:** NTK stays approximately constant during training (width $m \gg n$). The network behaves like a kernel method—features are frozen at initialization.
- **Feature learning:** NTK changes significantly during training (width $m = O(n)$). The network learns task-specific representations—features evolve during training.

Theorem: Global Convergence for Overparameterized Networks (Du et al. 2019; informal)

For sufficiently wide two-layer networks, under random initialization, nondegenerate training data, suitable activation/loss assumptions, and a small enough learning rate, gradient descent can converge to training loss $< \varepsilon$ with high probability.

Wrong Path: NTK Theory Explains All of Deep Learning**Why incomplete:**

1. Assumes lazy training (networks stay near initialization), but practical networks learn features
2. Requires impractically large width
3. Explains optimization but not generalization
4. Treats all architectures similarly, but architecture matters enormously in practice
5. Ignores the power of feature learning—the main advantage of deep learning over kernels!

What NTK does explain:

- Why very wide networks optimize easily (lazy regime)
- Connection between neural networks and kernel methods
- Lower bound on what networks can achieve (kernels $\subseteq$ neural networks)

What NTK misses:

- Why finite-width networks generalize better than kernels
- How networks learn hierarchical representations
- Why architecture design matters

Connection

Preview of Week 12: NTK explains why networks **optimize** (lazy training $\rightarrow$ kernel methods $\rightarrow$ PL condition $\rightarrow$ exponential convergence). But why do they **generalize**? Week 12 covers implicit regularization, compression, and modern phenomena (grokking, scaling laws).

Mental Model: The Width Spectrum

Understanding training across different widths:

Narrow networks ($m = O(1)$):

- Hard to optimize (many bad local minima)
- If optimization succeeds: good generalization (strong implicit bias)
- High variance (sensitive to initialization)

Moderate width ($m = O(n)$ to $O(n^2)$):

- Easier optimization (fewer bad local minima)
- Feature learning (NTK changes during training)
- Best generalization (sweet spot)
- This is the practical regime!

Infinite width ($m \rightarrow \infty$):

- Easiest optimization (linear dynamics, exponential convergence)
- No feature learning (lazy training)
- Kernel method behavior
- Explains optimization but misses generalization benefits

Key insight: Finite width is crucial for both tractable optimization AND good generalization through feature learning!

This leads us to the next question: NTK theory explains optimization convergence, but we still face the generalization puzzle. In the overparameterized regime, there are infinitely many solutions that perfectly fit the training data—why does gradient descent find one that generalizes well? The answer lies in **implicit regularization**: gradient descent has built-in biases that favor simple, generalizable solutions even without explicit penalties. This is perhaps the deepest insight in modern deep learning theory.